

Relazione progetto Ingegneria del Software

September 15, 2026

Giosuè Corbioli (VR518264)

Giovanni Grigoli (VR500669)

Rey Merzari (VR517101)

Denis Modenese (VR518328)

Dip. Informatica

Università degli studi di Verona

Contenuti

1	Introduzione	3
2	Class Diagram	4
2.1	Class Diagram della modellazione concettuale	5
2.2	Class Diagram del software	6
3	Use Case Diagram	7
4	Sequence Diagram	8
5	Activity Diagram	11
6	Stili Architetturali e Pattern	13
6.1	Stili Architetturali	13
6.2	Design Pattern	13
7	Testing	14
7.1	Unit Test	14
7.2	System Test	14

1 Introduzione

Il progetto da sviluppare scelto è quello relativo al servizio clinico per la gestione dei pazienti diabetici.

Esso richiede di sviluppare un sistema che permette ai pazienti di registrare le proprie rilevazioni di glicemia giornaliera ed eventuali sintomi annessi, oltre che seguire le terapie assegnate dai diabetologi, i quali possono anche visualizzare e modificare i dati dei pazienti.

Il sistema permette inoltre la comunicazione diretta tra le due parti e l'invio di alert se il paziente non segue la terapia stabilita.

In seguito sono elencati i diagrammi utilizzati per la progettazione, oltre che una spiegazione delle decisioni concettuali e architetturali prese.

2 Class Diagram

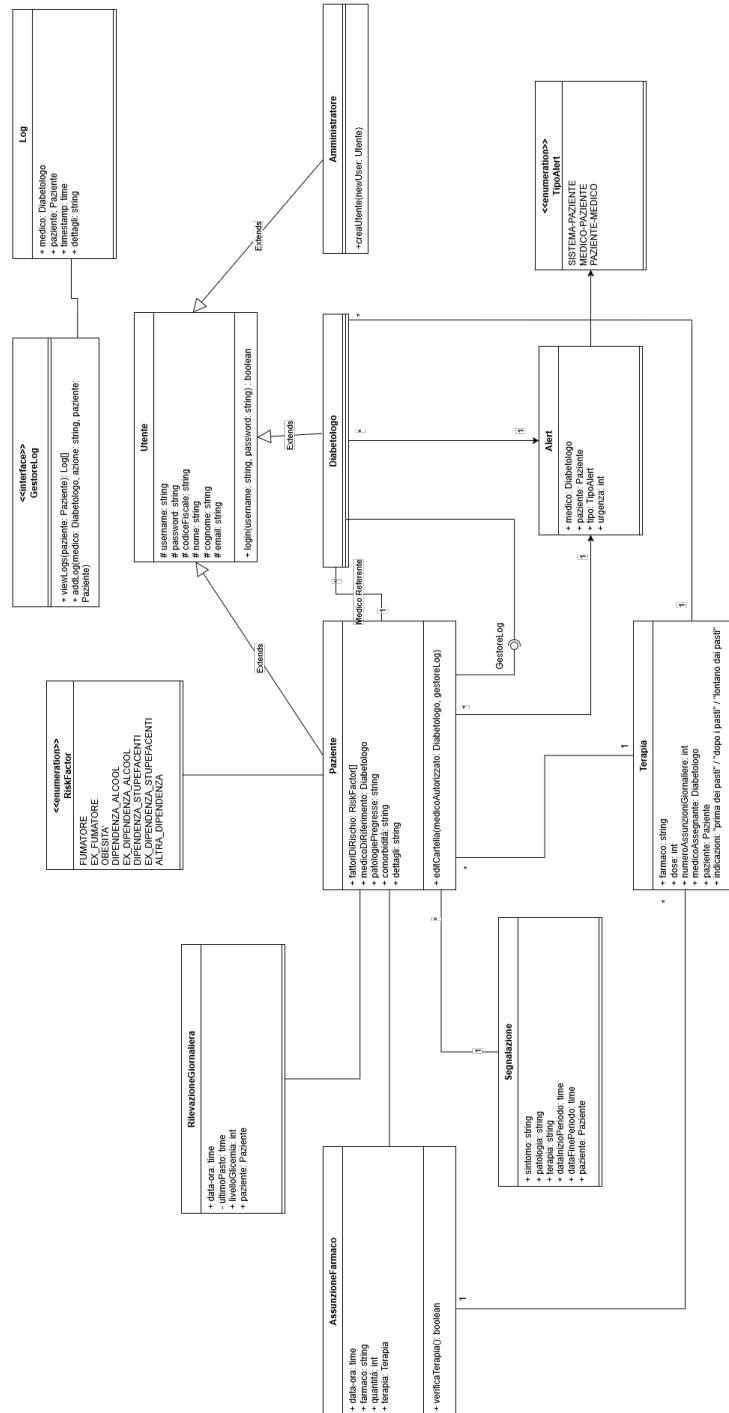


Figure 1: Class Diagram

2.1 Class Diagram della modellazione concettuale

Il sistema è incentrato sulla classe astratta *Utente*, raccoglie i dati comuni ai diversi tipi di utilizzatori, quali username, password, codice fiscale, nome, cognome ed email. Da essa derivano le classi *Paziente*, *Diabetologo* e *Responsabile*.

A ogni *Paziente* è associato un *Diabetologo* come medico di riferimento. Il *Paziente* mantiene inoltre i propri fattori di rischio, rappresentati tramite l'enumerazione *RiskFactor*, oltre che le proprie patologie pregresse, comorbidità e altri dettagli clinici. La modifica della cartella clinica (*editCartella()*) richiede l'autorizzazione di un medico e viene registrata tramite l'interfaccia *GestoreLog*, che salva ogni operazione in un oggetto *Log* (medico, paziente, timestamp, dettagli), in modo da assicurare il requisito di tracciabilità delle operazioni dei medici.

Le rilevazioni giornaliere registrano data e ora, ultimo pasto e livello di glicemia. Le assunzioni di farmaci sono verificate rispetto alla *Terapia* assegnata tramite *verificaTerapia()*.

Il paziente può inoltre effettuare *Segnalazioni* relative a sintomi, patologie o altre situazioni rilevanti. Il sistema gestisce messaggi tra paziente e diabetologo e genera messaggi di sistema destinati al paziente o al medico in presenza di condizioni che richiedono attenzione, come valori glicemici fuori soglia o mancata aderenza alla terapia.

Il *Diabetologo* può visualizzare i pazienti di propria competenza, consultare il loro andamento glicemico e gestire le relative terapie e informazioni cliniche. La classe *Terapia* contiene le informazioni relative al farmaco prescritto, alla dose, al numero di assunzioni giornaliere, al medico assegnante e alle indicazioni. Una stessa terapia può essere associata a più pazienti. Quando una terapia condivisa viene modificata per un singolo paziente, viene creata una nuova terapia per evitare di modificare quella degli altri pazienti.

Il *Responsabile* è incaricato della gestione degli utenti del sistema, in particolare della creazione e modifica dei dati relativi a pazienti e diabetologi.

La persistenza dei dati è garantita dalla classe *Database*, che centralizza il caricamento, il salvataggio, l'inserimento, la modifica e il recupero delle informazioni memorizzate nel file del database.

2.2 Class Diagram del software

TODO: ??????

3 Use Case Diagram

TODO: Fare gli use case e tagliarsi le vene. Ordine delle 2 a piacere

Paziente: login, registrare rilevazione glicemica, registrare sintomo, registrare assunzione farmaco, inviare messaggi al medico di riferimento, ricevere segnalazione, visualizzare cartella clinica, visualizzare terapia

Diabetologo: login, visualizzare dati paziente (anche in forma sintetica) e l'andamento della glicemia, assegnare e modificare terapia, aggiornare fattori di rischio, patologie e comorbidità dei pazienti, inviare messaggi ai pazienti, ricevere alert, consultare i log

Amministratore: login, creare nuovo utente nel Sistema, generare alert per soglie di glicemia superate o mancata assunzione farmaci per 3+ giorni

4 Sequence Diagram

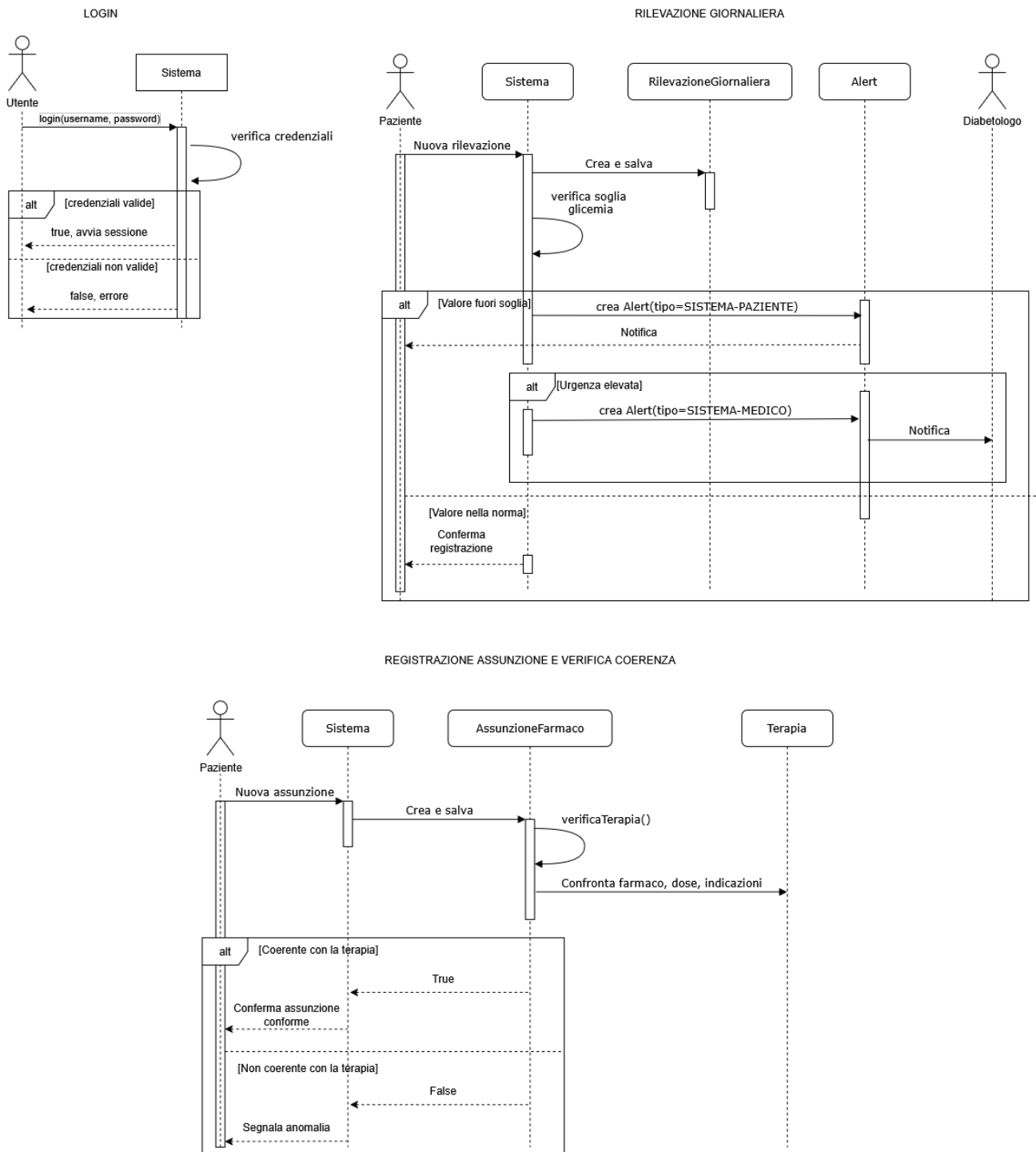


Figure 2: Sequence Diagram (Parte 1)

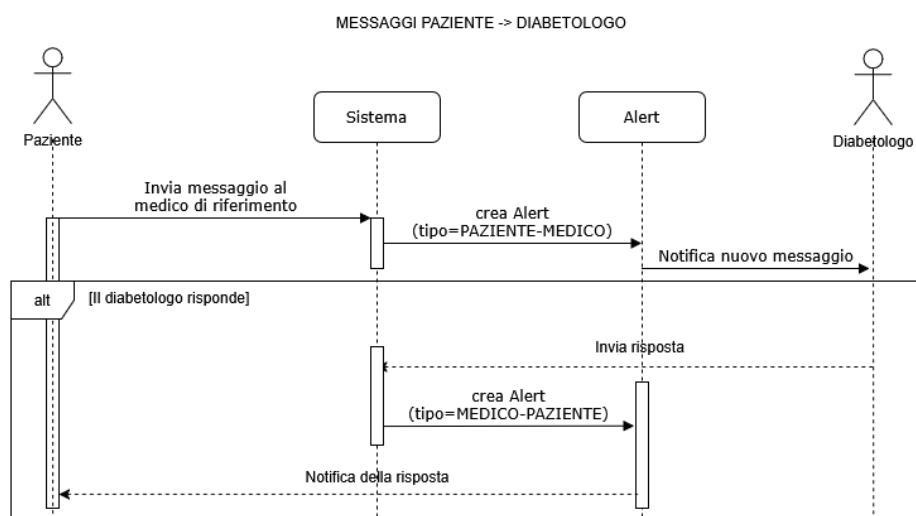
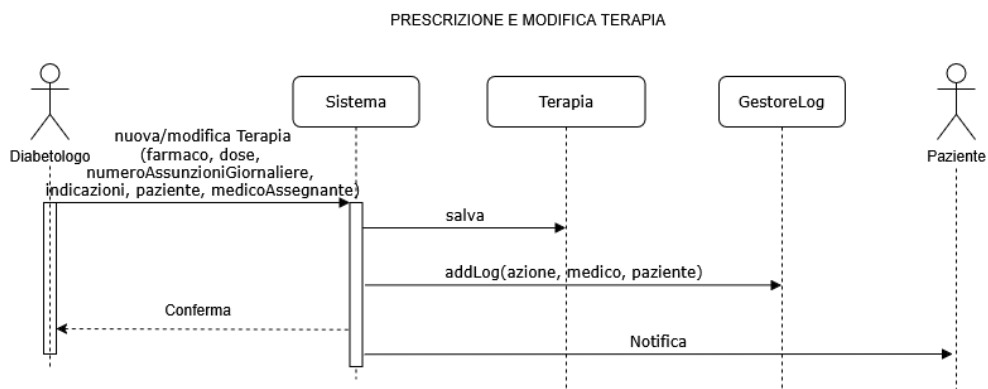
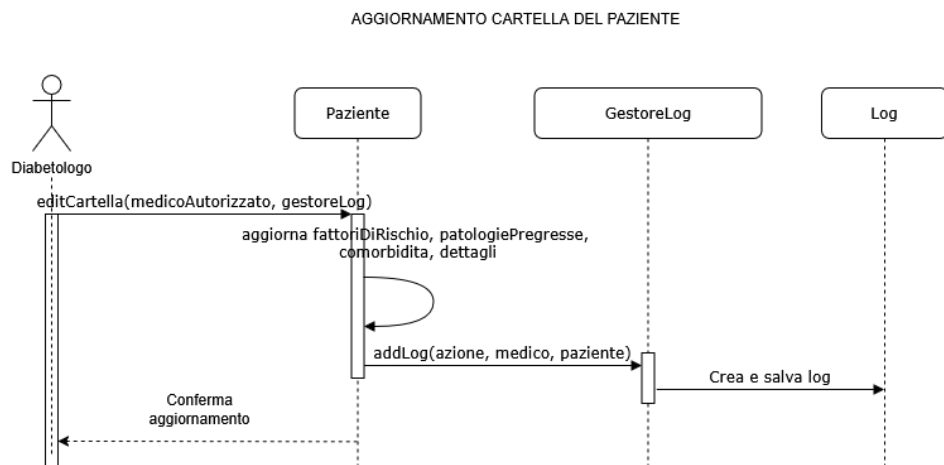


Figure 3: Sequence Diagram (Parte 2)

TODO: Spiegazione. Nel complesso o caso per caso?

5 Activity Diagram

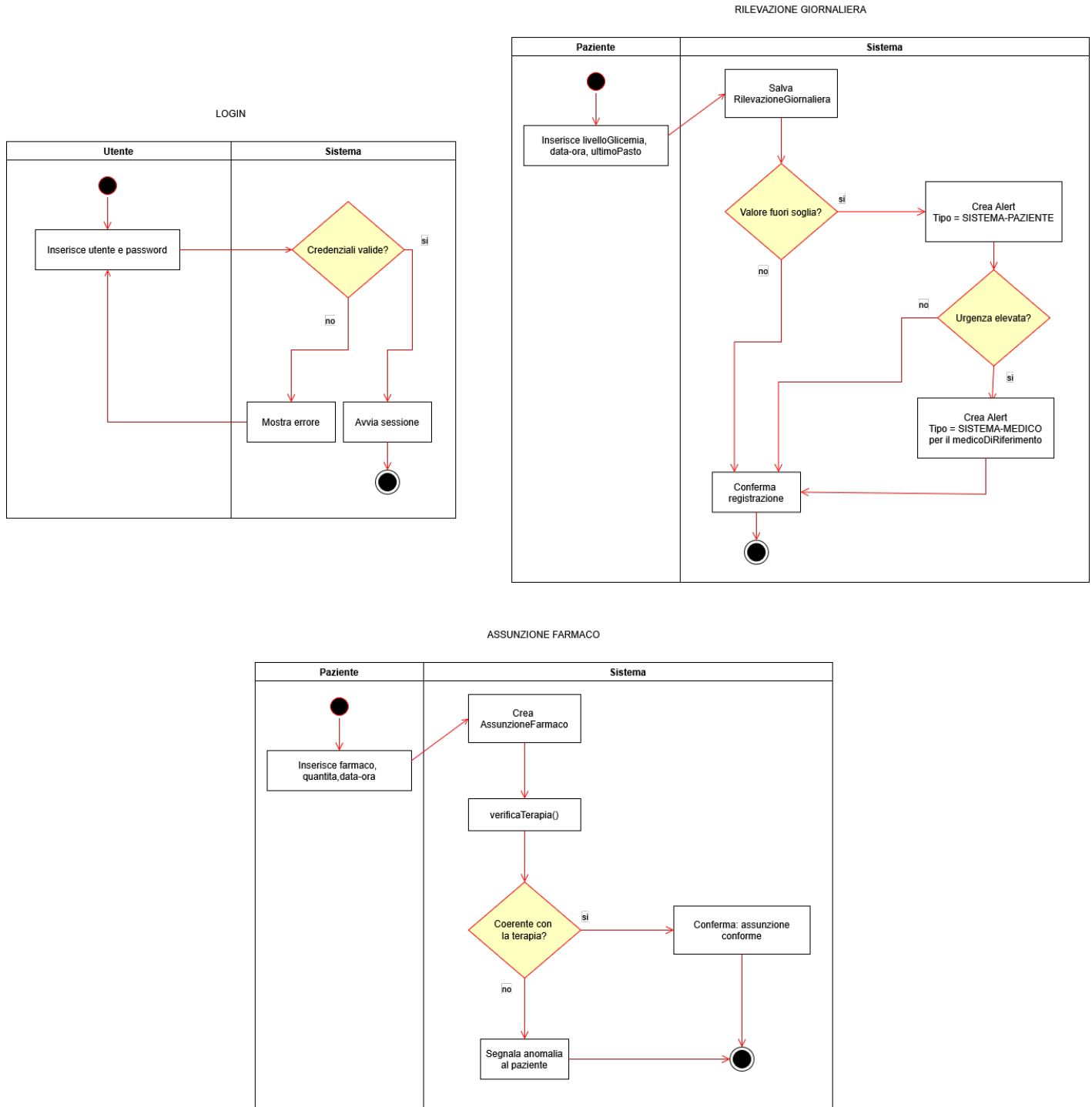
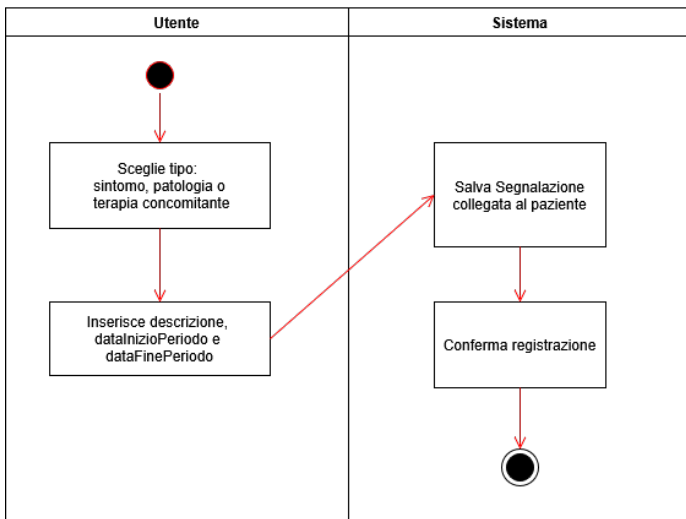
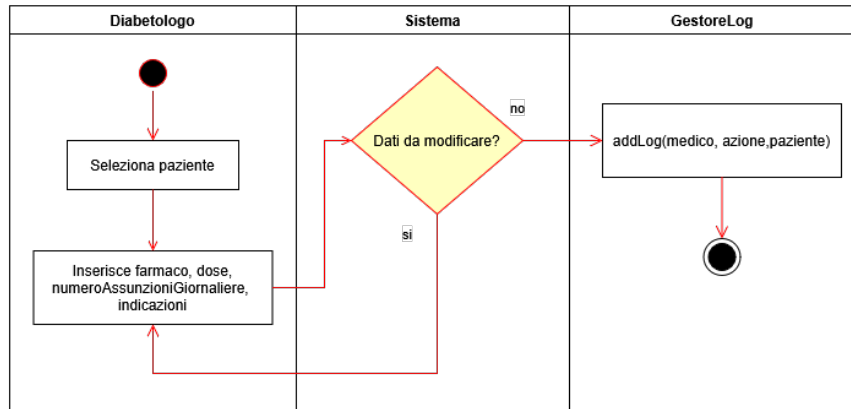


Figure 4: Activity Diagram (Parte 1)

REGISTRAZIONE SEGNALAZIONE



PRESCRIZIONE E MODIFICA DI UNA TERAPIA



AGGIORNAMENTO CARTELLA DEL PAZIENTE

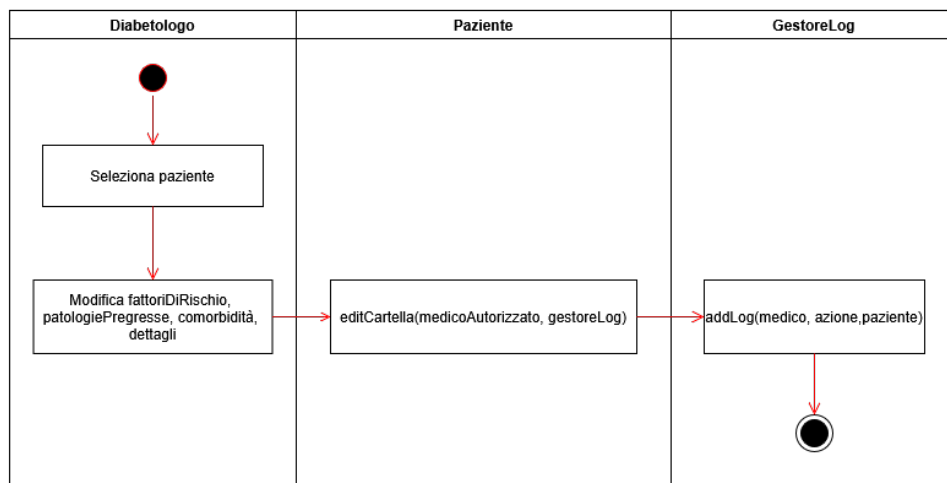


Figure 5: Activity Diagram (Parte 2)

TODO: Idem come il sequence

6 Stili Architetture e Pattern

6.1 Stili Architetture

Il software segue principalmente un'architettura a strati, nella quale le responsabilità sono suddivise tra interfaccia grafica, logica applicativa e gestione della persistenza.

Il Presentation Layer è costituito dalle viste JavaFX e dai relativi controller, che gestiscono l'interazione con l'utente e la visualizzazione dei dati.

Il Business Logic Layer comprende le classi del dominio, tra cui Utente, Paziente, Diabetologo, Terapia, Rilevazione, AssunzioneFarmaco, Segnalazione e le classi responsabili della gestione degli alert e delle regole applicative.

Il Data Access Layer è rappresentato dalla classe Database, che centralizza l'accesso ai dati persistenti. I controller e le altre classi che necessitano di recuperare o modificare informazioni persistenti utilizzano il Database come punto di accesso, evitando di gestire direttamente la scrittura sul file. La persistenza viene dunque realizzata tramite serializzazione Java degli oggetti all'interno del file database.data.

6.2 Design Pattern

Nel progetto è utilizzato il Design Pattern Singleton per la classe Database. Il sistema mantiene infatti una sola istanza condivisa del database, ottenuta tramite il metodo getInstance(). Questo permette di centralizzare l'accesso ai dati e di avere un unico punto di gestione della persistenza durante l'esecuzione dell'applicazione.

Il Database espone inoltre metodi specifici per il recupero e la modifica dei dati, ad esempio per ottenere i pazienti associati a un determinato diabetologo, le rilevazioni di un paziente o i messaggi destinati a un medico o a un paziente.

Per la gestione della GUI viene utilizzata la struttura controller-view di JavaFX, nella quale i controller gestiscono gli eventi e coordinano l'accesso ai dati necessari alle diverse schermate. **TODO: Log?**

TODO: Creazionali(Factory?)/Strutturali/Comportamentali

7 Testing

Il testing è stato realizzato sulla base dell'approccio White Box.

7.1 Unit Test

Sono stati realizzati test unitari relativi alla gestione della sessione e alla persistenza dei dati. Per la classe Session è stato verificato che il metodo `getInstance()` restituisca sempre la stessa istanza e che l'utente corrente venga correttamente impostato e rimosso tramite `setCurrentUser()` e `logout()`.

Per la classe Database sono stati testati l'inserimento senza duplicati di pazienti, diabetologi e responsabili, il recupero delle terapie, assunzioni, rilevazioni e segnalazioni associate a uno specifico paziente, il funzionamento del login con credenziali corrette e scorrette e la sostituzione delle rilevazioni tramite i metodi di aggiornamento. Sono inoltre stati verificati i metodi di recupero dei messaggi destinati a un determinato medico o paziente.

7.2 System Test

TODO: Tutto

(Test sull'applicazione completa)

Probabili casi da verificare:

- Registrazione di una glicemia fuori soglia -> alert > nella dashboard del diabetologo di riferimento
- Mancata registrazione di assunzioni farmaco per più di 3 giorni consecutivi -> alert generato al medico
- Modifica di una terapia da parte del medico -> le assunzioni successive vengono validate sulla nuova terapia
- Login con credenziali errate -> accesso rifiutato
- Impossibilità di accedere a dati altrui per chi non ne ha le facoltà